Optimal, Predictive, and Iterative Learning Control of Two Unstable Platforms

Nabil Al-Shanteer *University of Southampton*
Southampton, United Kingdom
Email: nas1u21@soton.ac.uk

***Abstract*—Two control problems are studied. In Part A, a PID and an MPC are designed to balance the NXTway-GS, an unstable two-wheeled robot whose tilt angle doubles every 93 ms when uncontrolled. The PID recovers a 5° initial tilt in 0.102 s but demands 17.81 V at peak. The MPC achieves the same recovery within 7.47 V and respects the voltage limit by construction. Under gyroscope noise the PID's derivative action drives rail-to-rail voltage chattering, while the MPC's internal Kalman estimator produces smooth commands under identical noise. A cascade PID extension bounds wheel-position drift to ± 2 rad. A two-output MPC with a 0.5 s prediction horizon regulates tilt and position inside a single optimisation. In Part B, LQR, MPC, and Iterative Learning Control are applied to the QUBE-Servo rotary inverted pendulum tracking a 45°, 5 s sinusoid. LQR and MPC produce similar responses with steady-state error norms of ~ 4 and ~ 2 respectively; their feedback gains differ only at the third decimal place, consistent with the unconstrained MPC gain converging to the LQR gain as the horizon grows. Two ILC variants are compared on the LQR-stabilised plant, the first of which is a PD-type update converges in ~ 14 iterations to a steady-state error norm of ~ 1, while a model-based norm-optimal update converges in ~ 3 iterations but settles at a higher steady-state error of ~ 2, exposing the trade-off between convergence speed and sensitivity to plant-model mismatch.**

I. Introduction

The NXTway-GS is a self-balancing, two-wheeled, programmable robot that platforms the control objective for the first half of this report II. Inherently unstable in the sense that a small tilt causes it to fall, feedback control is required and refers to the practice of continuously reading a sensor output, converting it to adjust a corrective action at the input. The objective is to maintain the robot in its upright equilibrium primarily through regulating its body tilt angle ψ to zero, and secondarily its wheel angle $t\theta$ to zero as well.

Stability is the property we desire our controller to provide. A stable system returns to its equilibrium after a disturbance, which we attempt to simulate in MATLAB's Simulink. We do this by taking two approaches which we later compare. The first of which uses a Proportional-Integral-Derivative (PID) controller that produces a corrective output (a motor voltage command) from three components acting on the tilt error. The proportional term scales the current error and provides an immediate response, the integral term sums error over time and eliminates any persistent offset that the proportional term cannot remove, and the derivative term estimates the rate of error change and counteracts overshoots before they develop. The second method uses Model Predictive Control (MPC). At each timestep, it uses a mathematical model of the robot to predict its future behaviour over a short prediction horizon. We use it to solve a constrained optimisation to find the optimal input sequence minimising predicted tracking error while staying under the motor voltage limit the hardware offers. Only the first input in that sequence is applied. At the next step the horizon shifts forward and the optimisation reiterates. This strategy is called the receding-horizon and lets MPC account for constraints in our hardware that the PID is incapable of doing.

The second part of this report moves from regulation to tracking, in Section III, with a comparison of three controllers on the Quanser QUBE-Servo configured as a rotary inverted pendulum. The control objective shifts to the upright balance of a pendulum while the arm angle follows a 45°, 5 s sinusoid. A Linear Quadratic Regulator (LQR) controller is used as a one-shot offline calculation minimising a quadratic cost over an infinite horizon, then unconstrained MCP is implemented using a finite-horizon lifted formulation whose closed-form gain converges to the LQR gain as the horizon grows, and lastly, Iterative Learning Control (ILC) that accumulates information across trials and uses the previous trial's error to "learn" to refine the input for the next trial. Two ILC variants are studied, a model-free PD-type update requiring no plant identification, and a model-based norm-optimal update using the identified plant matrix to compute optimal correction in one step. The two are compared directly on the same LQR-stabilised plant.

II. Part A – Lego Mindstorms NXT Robot

Linearisation and State-Space Form

The robot's dynamics come from the Euler-Lagrange equations, derived from the system's kinetic and potential energy. These equations are nonlinear in general because they contain $\sin\psi$ and $\cos\psi$ terms. Close to the upright equilibrium they can be approximated as linear by substituting $\sin\psi \approx \psi$ and $\cos\psi \approx 1$. This *small-angle linearisation* is valid for tilts up to roughly 15° and converts the dynamics into the *state-space form*

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}, \quad \mathbf{y} = \mathbf{C}\mathbf{x} \tag{1}$$

Here $\mathbf{x}$ is the *state vector*, the minimal set of variables that fully describes the system at any instant. $\mathbf{u}$ is the input, $\mathbf{y}$ is the measured output, and $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ are constant matrices derived from the physical parameters.

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -409.718 & -162.127 & 162.127 \\ 0 & 269.627 & 78.150 & -78.150 \end{bmatrix} \tag{2}$$

$$B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 157.580 & 157.580 \\ -75.958 & -75.958 \end{bmatrix} \tag{3}$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \tag{4}$$

Two decoupled subsystems emerge from the linearisation. Their equation sets share no variables. Subsystem $\mathcal{S}_1$ governs translation and tilt

$$\mathbf{x}_1 = [\theta,\ \psi,\ \dot{\theta},\ \dot{\psi}]^\top \tag{5}$$

where θ is wheel rotation angle (forward position), ψ is body tilt, and dots denote time derivatives. Subsystem $\mathcal{S}_2$ governs yaw and is independently stable. All control effort targets $\mathcal{S}_1$.

The instability has a single physical source. The matrix entry $A_{142} = gMLe_{11}/\det \mathbf{E}$ in $\mathbf{A}$ is positive because every factor, gravity g, body mass M, centre-of-mass height L, and inertia-matrix element e_{11}, is positive. This entry injects positive feedback from tilt into tilt acceleration. The more the robot leans, the faster it accelerates into the lean.

Open-Loop Poles and Observability

The *eigenvalues* of $\mathbf{A}$, also called poles, govern how each natural mode of the system evolves over time. A pole with a positive real part means that mode grows exponentially. A pole at zero means the corresponding variable drifts freely with no restoring force. MATLAB finds the poles:

$$\lambda(\mathbf{A}) = \{+7.44,\ 0,\ -6.52,\ -241.19\} \text{ rad/s} \tag{6}$$

The pole at $+7.44$ rad/s sets the 93 ms doubling time ($\ln 2/7.44$). The zero pole belongs to θ. Wheel position drifts freely whenever tilt error is zero and no outer loop drives it back. The two negative poles are stable decaying modes. Fig. 1 confirms the instability.

Observability determines whether all internal states can be reconstructed from the sensor output alone. A state is unobservable if it leaves no measurable imprint on the output. The first column of $\mathbf{A}$ is zero. Wheel position θ never influences any other state and leaves no fingerprint on the gyroscope. The observability rank from ψ alone is 3 out of 4, with θ structurally unobservable. A reduced three-state model $[\psi, \dot{\theta}, \dot{\psi}]^\top$ achieves full rank 3/3 and forms the plant for the MPC Kalman estimator.

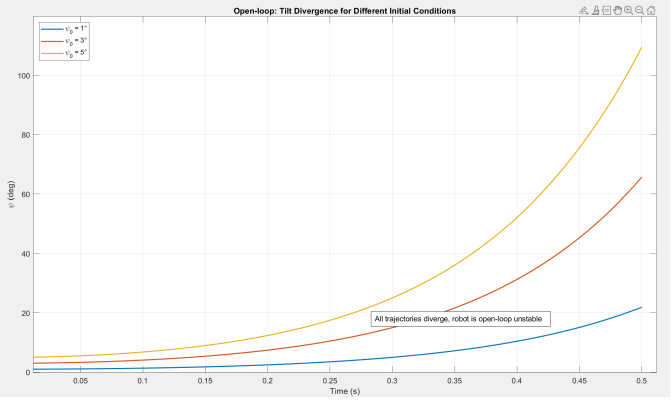


Fig. 1. Open-loop tilt divergence from three initial conditions.

PID Controller

For straight-line balancing both motor voltages are set equal ($V_L = V_R = V$), reducing the plant to a single-input single-output system from voltage V to tilt ψ. An integrator state ξ satisfying $\dot{\xi} = -\psi$ is appended, tracking accumulated tilt error and enabling the integral term. The PID implements as structured state feedback on the augmented state $\mathbf{x}_{\text{aug}} = [\theta, \psi, \dot{\theta}, \dot{\psi}, \xi]^\top$

$$u = -[0,\ K_p,\ 0,\ K_d,\ -K_i]\,\mathbf{x}_{\text{aug}} \tag{7}$$

We use MATLAB's `pidtune` to run with a target phase margin of 60°. The phase margin is a frequency-domain stability measure; the angle at which the open-loop phase response exceeds $-180°$ at the crossover frequency, which is the frequency where the loop gain equals one. A margin of 0° places the closed loop on the boundary of oscillation. Increasing it makes the response slower but more adaptive to modelling errors.

The tuned gains are $K_p = -204.14$, $K_i = -5474.04$, $K_d = -0.87$. A positive voltage drives the wheels forward so tilt feedback must produce a negative gain to oppose the error. The achieved phase margin is 79.2° at a crossover of $\omega_c = 119.45$ rad/s. The gain margin is infinite. The loop phase never reaches $-180°$, so no finite gain increase can destabilise the loop.

The closed-loop eigenvalues of the augmented system are $\{0,\ 0,\ -270.5,\ -51.2 \pm 20.2j\}$ rad/s. The two zero eigenvalues arise from θ's structural unobservability within the inner loop. They do not destabilise the system. The cascade outer loop corrects the drift they produce. The complex pair at $-51.2 \pm 20.2j$ rad/s gives a damped oscillatory mode at 3.2 Hz, which generates the small tilt undershoot visible in simulation.

Under ideal MATLAB conditions (no saturation applied), the controller recovers from $\psi_0 = 5°$ in 0.102 s with a 1° undershoot, demanding 17.81 V at peak. This exceeds the 9 V motor rating but passes in simulation. On hardware the ± 9 V saturation block (in Fig. 2) clamps the spike, reducing peak torque and extending settling time.

Fig. 2 shows the full Simulink implementation. The reference $\psi_{\text{ref}} = 0$ feeds into a summation block that subtracts the measured tilt to form the error. The PID block generates a voltage command, which the Motor Voltage Limit block

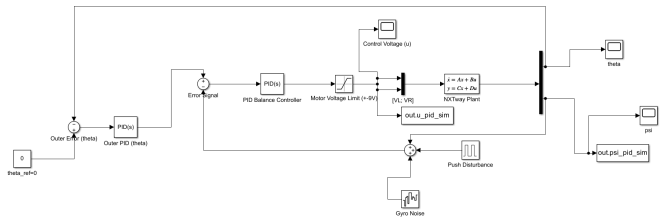


Fig. 2. Simulink diagram of the cascade PID. The inner PID loop stabilises tilt ψ. An outer proportional loop derives a tilt reference from wheel encoder θ to regulate position. Gyro Noise, a 20 ms sensor delay, and Push Disturbance simulate realistic operating conditions.

limits to $\pm 9\,\mathrm{V}$ before entering the NXTway Plant (the state-space model (1)). Gyro Noise adds band-limited white noise (power 10^{-7}, $T_s = 0.01\,\mathrm{s}$, giving $\approx \pm 0.003\,\mathrm{rad}$ amplitude) to simulate sensor imperfections. Push Disturbance injects three brief impulses. We use an outer cascade loop to read wheel position θ and feed a corrective tilt reference back into the inner balance controller.

A cascade architecture addresses the position drift inherent in the single-loop PID. The base PID controls only ψ. Once $\psi = 0$ the integral state ξ settles to a constant and the motor voltage drops to zero, leaving the wheels wherever they stopped. Over a 30 s run this produces $-200\,\mathrm{rad}$ of uncorrected drift. The cascade adds a second, slower outer loop that reads θ, computes the position error, and outputs a tilt reference ψ_{ref}. A forward displacement commands a slight backward lean, driving the wheels backwards. The inner loop is faster than the outer, so balance is maintained while position slowly gets corrected.

Fig. 5 shows the 30 s Simulink result. With outer gain $K_\theta = 0.03$, wheel position stays within $\pm 2\,\mathrm{rad}$ throughout. Tilt sustains a $\pm 0.07\,\mathrm{rad}$ oscillation aided by the outer loop continuously updating ψ_{ref}. Adding integral or derivative terms to the outer loop destabilised the system in all tested configurations. The inner K_d amplifies every step change in ψ_{ref} and the outer dynamics push the combined loop past stability.

MPC Controller

MPC frames the control problem as a finite-horizon constrained optimisation. At each time step k, the following quadratic programme (QP) is solved:

$$\min_{u} \sum_{j=1}^{H_p} Q_y\, e_y^2(k+j) + \sum_{j=0}^{H_c-1} \left[R_u\, u^2(k+j) + R_{\Delta u}\, \Delta u^2(k+j) \right]$$

subject to $-9\,\mathrm{V} \le u(k+j) \le +9\,\mathrm{V}$ for all j, where $e_y = \psi_{\mathrm{ref}} - \psi$, H_p is the prediction horizon, H_c is the control horizon, and Q_y, R_u, $R_{\Delta u}$ are tuning weights. Q_y penalises tracking error, R_u penalises voltage magnitude, and $R_{\Delta u}$ penalises abrupt voltage changes. The voltage constraint is enforced every step, and unlike PID, the voltage limit is not violated by construction. Only the first input of the solved sequence is applied. The horizon shifts by one step and the QP is solved again.

The continuous-time plant is converted to discrete time via zero-order hold (ZOH) at $T_s = 0.01\,\mathrm{s}$ (100 Hz). ZOH holds each voltage command constant for one sample interval. This is a valid approximation at 100 Hz because the dominant robot dynamics operate near 7.44 rad/s (about 1.2 Hz), well below the sample rate.

[b]0.9



Fig. 3. Tilt ψ (rad)

[b]0.9

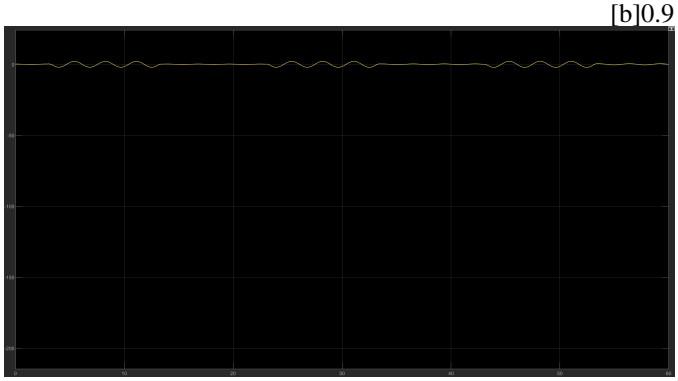


Fig. 4. Position θ (rad)

Fig. 5. Cascade PID: 30 s Simulink run with noise, delay, and push disturbances. Tilt (a) oscillates within ± 0.07 rad. Position (b) is bounded to ± 2 rad, vs. -200 rad drift without the outer loop.

TABLE I
MPC CONTROLLER PARAMETERS

Param.	**Value**	**Rationale**
H_p	20 steps (0.2 s)	Covers $\approx 1.5\times$ the unstable mode time constant ($1/7.44 \approx 0.13$ s)
H_c	5 steps (0.05 s)	Reduces decision variables; smooths actuation
Q_y	10	Strongly penalise tilt deviation
R_u	0.1	Allow large voltages when needed
$R_{\Delta u}$	0.2	Suppress rapid switching

The unstable mode has a time constant $\tau = 1/7.44 \approx 0.13\,\mathrm{s}$. The prediction horizon $H_p = 50$ steps (0.5 s) covers 3.7 of these time constants, giving the optimiser enough look-ahead to plan a coordinated correction. A shorter 0.2 s horizon cannot see wheel-position error accumulating. Over 0.5 s it can, and it plans the position correction alongside tilt recovery. The control horizon $H_c = 5$ steps (0.05 s) reduces the number of free decision variables in the QP and enforces smooth actuation. Output weights are $Q_y = \mathrm{diag}(5, 10)$, penalising tilt twice as heavily as position.

An internal Kalman filter reconstructs all plant states at each step from the noisy sensor readings. A Kalman filter is a recursive algorithm that blends two information sources.

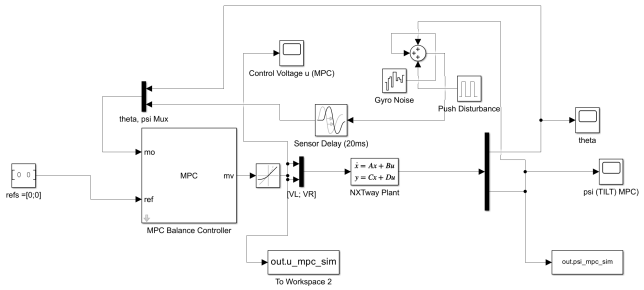


Fig. 6. Simulink diagram of the two-output MPC. Noise, delay, and push disturbances match the PID model.

It weights a model prediction (what the physics says the state should be) against the incoming sensor measurement (what the sensor actually reports). A very noisy sensor leads the filter to trust the model more heavily, and vice versa. The result is a clean state estimate at every step, even when the raw sensor is corrupted. This is why the MPC voltage remains smooth under the same noise conditions that drive the PID to rail-to-rail chattering.

Fig. 6 shows the two-output MPC Simulink model. The MPC block receives both tilt ψ and wheel position θ through the `mo` (measured output) port alongside a joint reference `refs` = $[0; 0]$. It runs the Kalman filter and solves the QP internally at each step, outputting a single voltage command `mv`. A Sensor Delay block introduces a 20 ms lag. Gyro Noise and Push Disturbance are identical to the PID model for a fair comparison.

The MPC handles tilt and position inside a single optimisation, no cascade needed. The 0.5 s prediction horizon lets the optimiser see position error developing and plan a coordinated response. A brief intentional lean drives the wheels toward the target. Once position is corrected the lean is removed.

Fig. 9 shows the 30 s Simulink result. Tilt ψ remains within ± 0.01 rad between disturbances, with brief recovery spikes at each push (at which it increases to $\pm 0.1 - 0.19$ rad). Wheel position θ returns to within ± 0.05 rad after each disturbance. The large initial excursion in θ at $t = 0$ reflects recovery from the 5° starting tilt, to which the controller leans the robot to drive the wheels forward. Both quantities remain stable and bounded throughout the full run.

Discussion

The PID settles fast. Under ideal conditions it returns the robot to upright in 0.102 s and tolerates infinite gain increases without destabilising. The main limitation is voltage. The tuning demands 17.81 V at peak, limited by the motor's 9 V rating. On hardware the clamp cuts peak torque and extends settling. The derivative gain that gives speed in ideal conditions in this case amplifies gyroscope noise into rail-to-rail chattering. The cascade outer loop corrects position drift but introduces a persistent limit cycle in tilt, and adding outer-loop dynamics to remove it was not possible without destabilising the combined system.

The MPC cannot exceed ± 9 V by construction. Its peak in the same recovery test is 7.47 V, well within the motor

[b]0.48

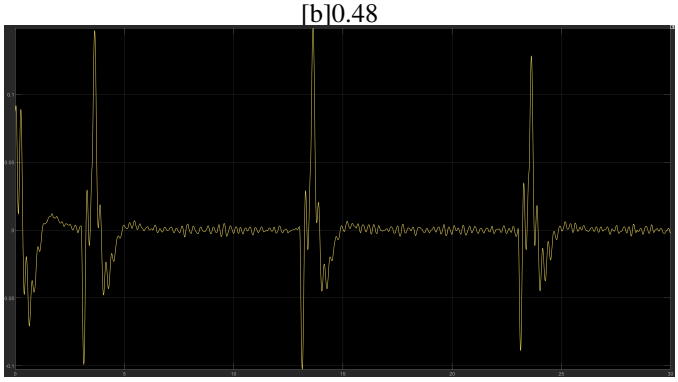


Fig. 7. Tilt ψ (rad)

[b]0.48

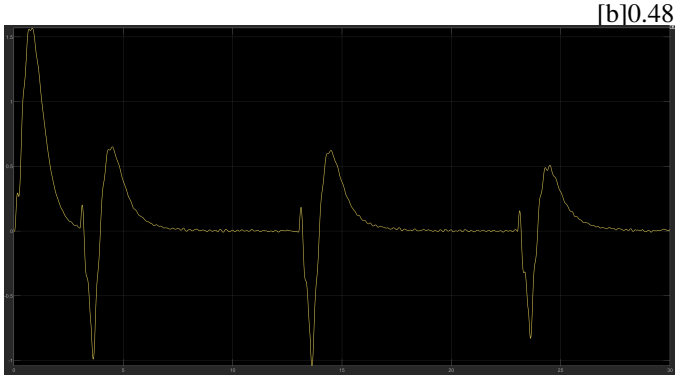


Fig. 8. Position θ (rad)

Fig. 9. Two-output MPC: 30 s Simulink run with noise, delay, and push disturbances. Tilt (a) stays within ± 0.1 rad. Position (b) returns to within ± 0.05 rad after each push.

rating. The Kalman estimator filters noise before it reaches the voltage command, so the Simulink response under sensor noise and disturbances is smooth. Position regulation sits inside the same optimisation as tilt control. No second tuning loop is needed. The cost is computation. A QP must be solved at every 10 ms step, which is feasible on modern embedded processors but impractical on the original NXT hardware without pre-computation or a faster platform.

Fig. 10 places both tilt responses on the same axes from the same 5° initial condition. The PID recovers marginally faster in ideal simulation, the MPC voltage stays inside the motor's physical limit throughout. On hardware, where the PID's 17.81 V peak is clipped to 9 V and the MPC's 7.47 V peak is not, the speed advantage reverses.

Part B replaces the NXTway-GS with the QUBE-Servo 2 in the pendulum configuration. The state vector expands to $[\theta, \alpha, \dot{\theta}, \dot{\alpha}]^\top$, where α is pendulum angle and θ is arm angle. LQR computes a full-state feedback gain matrix by minimising a quadratic cost over infinite time, a one-shot offline calculation that produces fast and smooth responses at zero online computation cost. MPC is retuned for the new plant with $H_p = 30$, $H_c = 10$, $T_s = 0.02$ s. ILC learns from repeated trials. Each trial's tracking error is used to update the next trial's feedforward command, converging to near-zero tracking error over 25 iterations without requiring a real-time plant model.

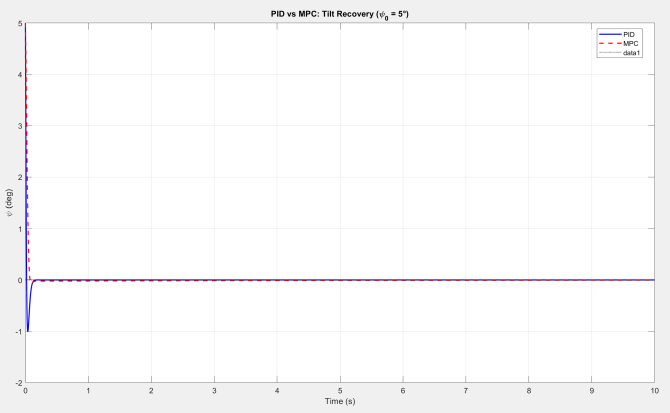


Fig. 10. PID versus MPC tilt recovery from $\psi_0 = 5°$ under ideal MATLAB conditions. The PID recovers in 0.102 s but needs 17.81 V at peak. The MPC peak is 7.47 V, within the motor rating, realistically suitable.

III. Part B – Quanser QUBE-Servo with NI myRIO

Linear Quadratic Control

Linear Quadratic Regulator (LQR) control refers to an optimal state-feedback design method for linear systems. Given the discrete-time model:

$$x_{t+1} = Ax_t + Bu_t, \quad y_t = Cx_t \tag{8}$$

With state $x_t \in \mathbb{R}^n$ and input $u_t \in \mathbb{R}^m$, the finite-horizon LQR objective is to find $u_0^*, \ldots, u_{N-1}^*$ that minimises the quadratic performance index:

$$J = \sum_{t=0}^{N-1} \left(x_t^\top Q x_t + u_t^\top R u_t\right) + x_N^\top Q_f x_N \tag{9}$$

With $Q, Q_f \succeq 0$ and $R \succ 0$. The matrices Q and Q_f penalise state deviation from the origin, while R penalises control effort; their magnitudes relative to one another allow the designer to trade-off between regulation performance and actuator usage.

The solution can be obtained via dynamic programming. By Bellman's principle of optimality, any tail of an optimal trajectory is itself optimal from the state it begins in; the cost-to-go takes the quadratic form $V_t(x_t) = x_t^\top P_t x_t$, where $P_t \succeq 0$ satisfies the discrete-time Riccati recursion:

$$P_t = Q + A^\top P_{t+1} A - A^\top P_{t+1} B \left(R + B^\top P_{t+1} B\right)^{-1} B^\top P_{t+1} A \tag{10}$$

Solved backwards from $P_N = Q_f$. The optimal control law is the linear state feedback:

$$u_t^* = K_t x_t, \quad K_t = -\left(R + B^\top P_{t+1} B\right)^{-1} B^\top P_{t+1} A. \tag{11}$$

For long enough horizons, P_t converges to a steady-state value P satisfying the discrete-time algebraic Riccati equation (DARE), yielding the constant gain:

$$K = -\left(R + B^\top P B\right)^{-1} B^\top P A \tag{12}$$

The control law is $u_t^* = Kx_t$. This is the infinite-horizon LQR and is synonymous to letting $N \to \infty$ in (9). Provided (A, B) is stabilisable and $(A, Q^{1/2})$ is detectable, $A + BK$ is Hurwitz, thus the method applies to open-loop unstable plants such as the inverted pendulum considered in this half of the report. In practice, P is computed directly via standard solvers like MATLAB's `lqr`.

Model Predictive Control

Model Predictive Control (MPC) reiterates the optimal control problem over a moving finite window. Typically, MPC is favoured by designers who want to enforce input and state constraints. This work uses the unconstrained variant, for which the optimiser admits a closed-form solution and no online solver is required, as opposed to the constrained version, which uses online optimisation via MATLAB's `quadprog` solver.

In our unconstrained formulation, the optimal first input $u_{t_0}^*$ takes on a closed-form linear state-feedback law computed once. At time t_0 with measured state x_{t_0}, the predicted state trajectory $\vec{x}_{t_0} = (x_{t_0}^\top, \ldots, x_{t_0+N-1}^\top)^\top$ is a linear function of x_{t_0} and the stacked input sequence $\vec{u}_{t_0} = (u_{t_0}^\top, \ldots, u_{t_0+N-1}^\top)^\top$,

$$\vec{x}_{t_0} = G\vec{u}_{t_0} + Hx_{t_0} \tag{13}$$

Where G and H are block matrices constructed from powers of A and B. Substituting (13) into the finite-horizon version of (9) gives a quadratic in $\vec{u}_{t_0}$:

$$J(\vec{u}_{t_0}) = \vec{u}_{t_0}^\top \left(\bar{R} + G^\top \bar{Q} G\right)\vec{u}_{t_0} + 2x_{t_0}^\top H^\top \bar{Q} G \vec{u}_{t_0} + \text{const.} \tag{14}$$

with block-diagonal weights $\bar{Q} = \mathrm{diag}(Q, \ldots, Q)$ and $\bar{R} = \mathrm{diag}(R, \ldots, R)$. Setting $\partial J / \partial \vec{u}_{t_0} = 0$ and extracting the first block gives:

$$u_{t_0}^* = Kx_{t_0}, \quad K = -[I, 0, \ldots, 0]\left(\bar{R} + G^\top \bar{Q} G\right)^{-1} G^\top \bar{Q} H \tag{15}$$

That is a fixed linear state-feedback law analogous to (12) but derived over a finite horizon. (15) is a finite-horizon optimum approaching the LQR gain only as $N \to \infty$, with N, Q, and R as the tuning parameters. Longer horizon estimates the infinite-horizon LQR more closely, but at the cost of more computations per sample. Tracking is handled similarly to LQR, replacing x_t with $\tilde{x}_t = x_t - x_{\text{ref}}$ in (14).

Iterative Learning Control

Iterative learning control is a scheme for improving the control input by exploiting a repeating reference signal. As the name suggests, the control input is successively refined by "learning" from previous iterations. A generic, model-free update law can be written as [1]:

$$\mathbf{u}_{k+1}(t) = Q(\sigma)[\mathbf{u}_k(t) + L(\sigma)\mathbf{e}_k(t+1)] \tag{16}$$

Where t is time, k is the iteration, and σ is the forward time-shift $qx(t) := x(t+1)$. The error is defined as $\mathbf{e}_k = \mathbf{y}_k - \mathbf{r}$ where $\mathbf{r}$ is the reference trajectory that the plant input wants to track. $Q(\sigma)$ is defined as the Q-filter [1] and can be chosen to improve learning stability. $L(\sigma)$ is the learning function (or gain).

There are many possible choices of both Q and L, and it would be useful to have a way to verify that the system that

describes the error behaviour is stable. We define our plant dynamics in the lifted form:

$$\underbrace{\begin{bmatrix} y_k(1) \\ \vdots \\ y_k(N) \end{bmatrix}}_{:=\mathbf{y}_k} = \underbrace{\begin{bmatrix} g(0) & 0 & \dots & 0 \\ g(1) & g(0) & & \vdots \\ \vdots & & \ddots & 0 \\ g(N-1) & g(N-2) & \dots & g(0) \end{bmatrix}}_{:=P/T_s} \mathbf{u}_k + \underbrace{\begin{bmatrix} d(1) \\ d(2) \\ \vdots \\ d(N) \end{bmatrix}}_{:=\mathbf{d}} \tag{17}$$

where $g(\cdot)$ is the impulse response of the system and $d(\cdot)$ is the free response. The system 17 with the ILC update equation 16 is stable if and only if [2]:

$$\rho(Q(I - LP)) < 1 \tag{18}$$

Where $\rho(\cdot)$ is the spectral radius ($\rho(\cdot) \equiv \max(|\text{eig}(\cdot)|)$). The error propagation model can be derived by substituting (16) into the lifted form over a trial $\mathbf{y}_{k+1} = P\mathbf{u}_{k+1}$. (19) is derived in Appendix A.

$$\mathbf{e_{k+1}} = (I - PQL)\mathbf{e}_k + (I - PQ)\mathbf{r} \tag{19}$$

The largest singular value of a matrix ($\bar{\sigma}(\cdot)$ denotes the largest singular value) can be used to bound the gain of the matrix [2]:

$$||F\mathbf{x}||_2 \leq \hat{\sigma}(F)||\mathbf{x}||_2 \tag{20}$$

Therefore, the error 2-norm $||\mathbf{e}_k||_2$ will monotonically converge if:

$$\hat{\sigma}(I - PQL) < 1 \tag{21}$$

Note that this does not necessarily show convergence to zero, but monotonic convergence to a steady-state error $||\mathbf{e}_\infty||_2$.

One choice of update law (and the one used in this project) is the PD-type ILC update law, defined as:

$$\mathbf{u}_{k+1}(t) = \alpha\mathbf{u}_k(t) + k_p\mathbf{e}_k(t+1) + k_d\dot{\mathbf{e}}_k(t+1) \tag{22}$$

Where the error derivative is estimated with the forward-time approximation $\dot{\mathbf{e}}_k \approx \mathbf{e}_k(t+1) - \mathbf{e}_k(t)$. Unlike a P-type-only ILC update, the derivative term helps prevent oscillations in the update (it effectively acts as a damper) and can accelerate convergence while maintaining a simple, low-computational-overhead implementation.

Another choice of update law (which we compare in this project) is the norm-optimal ILC (NOILC) update law [3]:

$$\mathbf{u^*_{k+1}} = \text{argmin}_{\mathbf{u_{k+1}}}\{q||\mathbf{e_{k+1}}||^2 + ||\mathbf{u_{k+1}} - \mathbf{u_k}||^\mathbf{2}\} \tag{23}$$

Computed at each iteration to give the solution:

$$\mathbf{u_{k+1}} = \mathbf{u_k} + (\mathbf{I} + q\mathbf{P^T}\mathbf{P})^{-1}q\mathbf{P^T}\mathbf{e_k} \tag{24}$$

Substituting the optimal update into $\mathbf{e}_{k+1} = \mathbf{e}_k - P(\mathbf{u}_{k+1} - \mathbf{u}_k)$ and applying the push-through identity gives $\mathbf{e}_{k+1} = (I + qPP^\top)^{-1}\mathbf{e}_k$. The contraction operator has all eigenvalues in $(0, 1]$ for any $q > 0$, so the error is monotonically non-increasing and the stability condition from (18) is satisfied. Compared with the PD-type law, NOILC reduces the tuning from three parameters to one scalar q, but requires the plant model P to be explicitly included in the controller.

Task B.1 – Modelling the System Dynamics

The Quanser QUBE-Servo is configured as a rotary inverted pendulum. We chose this over the inertial disc because balancing its unstable open-loop dynamics is a much tougher test for the controllers. The rotary arm (length L_r) is actuated by a DC servo motor and rotates in the horizontal plane; the pendulum (length L_p) is attached at the arm tip and swings freely in the vertical plane. The system has two degrees of freedom: the arm angle θ and the pendulum angle α, with a single input V_m (motor voltage). Following the convention in [4], $\alpha = 0$ corresponds to the pendulum hanging downward, increasing positively counter-clockwise. Both θ and α are measured directly by optical encoders.

The physical parameters are: pendulum mass $m_p = 0.024\,\text{kg}$, arm length $L_r = 0.085\,\text{m}$, pendulum length $L_p = 0.129\,\text{m}$, arm inertia $J_r = 5.72 \times 10^{-5}\,\text{kg m}^2$, pendulum inertia about its centre of mass $J_p = 1.28 \times 10^{-4}\,\text{kg m}^2$, motor resistance $R_m = 8.4\,\Omega$, and motor constant $k_m = 0.042$ (serving as both torque and back-EMF constant). The viscous damping coefficients $D_r = 0.0015\,\text{N m s/rad}$ and $D_p = 0.0005\,\text{N m s/rad}$ were obtained from [4]. The motor torque is:

$$\tau = \frac{k_m(V_m - k_m\dot{\theta})}{R_m} \tag{25}$$

Since the control objective is to balance the pendulum upright, the nonlinear equations of motion [4] are linearised about $\alpha = \pi$. Substituting $\alpha = \pi + \tilde{\alpha}$ with $\tilde{\alpha}$ small ($\sin\alpha \approx -\tilde{\alpha}$, $\cos\alpha \approx -1$) and neglecting second-order terms yields:

$$(m_pL_r^2 + J_r)\ddot{\theta} - \tfrac{1}{2}m_pL_pL_r\ddot{\tilde{\alpha}} = \tau - D_r\dot{\theta} \tag{26}$$

$$-\tfrac{1}{2}m_pL_pL_r\ddot{\theta} + (J_p + \tfrac{1}{4}m_pL_p^2)\ddot{\tilde{\alpha}} - \tfrac{1}{2}m_pL_pg\tilde{\alpha} = -D_p\dot{\tilde{\alpha}} \tag{27}$$

The gravity term $-\frac{1}{2}m_pL_pg\tilde{\alpha}$ reinforces deviations from vertical, confirming the unstable equilibrium. The off-diagonal coupling terms change sign relative to the downward linearisation in [4] due to $\cos\pi = -1$.

Substituting τ into (26) and grouping the $\dot{\theta}$ damping terms, the system can be written in matrix form as:

$$\mathbf{M}\begin{bmatrix}\ddot{\theta} \\ \ddot{\tilde{\alpha}}\end{bmatrix} = \begin{bmatrix}-\gamma & 0 \\ 0 & -D_p\end{bmatrix}\begin{bmatrix}\dot{\theta} \\ \dot{\tilde{\alpha}}\end{bmatrix} + \begin{bmatrix}0 \\ \frac{1}{2}m_pL_pg\end{bmatrix}\tilde{\alpha} + \begin{bmatrix}\frac{k_m}{R_m} \\ 0\end{bmatrix}V_m \tag{28}$$

where $\gamma = k_m^2/R_m + D_r$ combines the back-EMF and mechanical damping on $\dot{\theta}$, and the mass matrix is:

$$\mathbf{M} = \begin{bmatrix}m_pL_r^2 + J_r & -\frac{1}{2}m_pL_pL_r \\ -\frac{1}{2}m_pL_pL_r & J_p + \frac{1}{4}m_pL_p^2\end{bmatrix} \tag{29}$$

Inverting $\mathbf{M}$ and multiplying through yields the acceleration expressions. Defining $J_T = \det(\mathbf{M}) = J_pm_pL_r^2 + J_rJ_p + \frac{1}{4}J_rm_pL_p^2$ and the state $\mathbf{x} = [\theta,\, \tilde{\alpha},\, \dot{\theta},\, \dot{\tilde{\alpha}}]^T$, the state-space model $\dot{\mathbf{x}} = A\mathbf{x} + Bu$ is:

$$A = \begin{bmatrix}0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{m_p^2L_p^2L_rg}{4J_T} & -\frac{(J_p + \frac{1}{4}m_pL_p^2)\gamma}{J_T} & -\frac{m_pL_pL_rD_p}{2J_T} \\ 0 & \frac{m_pL_pg(J_r + m_pL_r^2)}{2J_T} & -\frac{m_pL_pL_r\gamma}{2J_T} & -\frac{(J_r + m_pL_r^2)D_p}{J_T}\end{bmatrix} \tag{30}$$

$$B = \begin{bmatrix} 0 \\ 0 \\ \frac{(J_p + \frac{1}{4} m_p L_p^2) k_m}{J_T R_m} \\ \frac{m_p L_p L_r k_m}{2 J_T R_m} \end{bmatrix} \tag{31}$$

Both encoders directly measure θ and α, so the output equation is:

$$\mathbf{y} = C\mathbf{x} + Du, \quad C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad D = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{32}$$

In practice the measured α is offset by π from $\tilde{\alpha}$, so the controller receives $\tilde{\alpha} = \alpha - \pi$. The angular velocities $\dot{\theta}$ and $\dot{\tilde{\alpha}}$ are not directly measured and must be reconstructed by a state observer.

Task B.2 – LQR Control

The LQR controller was designed based on the system dynamics derived in section III, with the gains calculated in MATLAB.

Since keeping the pendulum upright is the most important, we decided on choosing a state cost matrix of:

$$Q = \begin{bmatrix} 80 & 0 & 0 & 0 \\ 0 & 100 & 0 & 0 \\ 0 & 0 & 0.1 & 0 \\ 0 & 0 & 0 & 0.1 \end{bmatrix} \tag{33}$$

And our control cost $R = 1.5$. By using MATLAB's `lqr` function, our LQR gain was $K = [-6.3767, 36.0384, -1.8937, 3.3228]^\top$. This places the closed-loop poles at $\lambda_i \in \{0.5956, 0.96676, 0.9917 \pm 0.0394j\}$ and hence the system is stable.

For the LQR controller (and all subsequent controllers) we need accurate estimation of all four state variables. θ and α can be measured directly from the encoder angles. The derivatives of these ($\dot{\theta}$ and $\dot{\alpha}$) can't be directly obtained, and a true differentiator would not be physically realisable, and the noise in the measurements would lead to huge estimated derivatives. Instead, we chose the transfer function:

$$H(s) = \frac{100s}{s + 100} \tag{34}$$

This approximates a differentiator at low frequencies while bounding the gain at high frequencies. By using two of these "differentiators", we are able to accurately estimate all four state variables.

For tracking the $\theta \approx \theta_{\text{ref}}$ reference point, we define $\mathbf{x}_{\text{ref}} = [\theta_{\text{ref}}, 0, \dot{\theta}_{\text{ref}}, 0]^\top$ as the reference point and pass $\tilde{\mathbf{x}} = \mathbf{x} - \mathbf{x}_{\text{ref}}$ in the state-feedback. Therefore, our control law becomes:

$$u = -K\tilde{\mathbf{x}} \tag{35}$$

Once implemented on the pendulum, we set the reference trajectory to be a sinusoid of amplitude 45 degrees and period of 5 seconds.

Performance was good, and after a couple of seconds, the pendulum tracked the reference consistently, but with a constant error on each cycle. Figure 12 shows the error norm over time. The initial jump in error was due to the pendulum's initial stabilisation: it was hanging down (the final value of the free response) to calibrate the sensors, and needed to be pushed up and left to stabilise. After a few seconds (and once the pendulum angle is under control), the error drops to its steady state value with the consistent LQR response shown in figure 11.

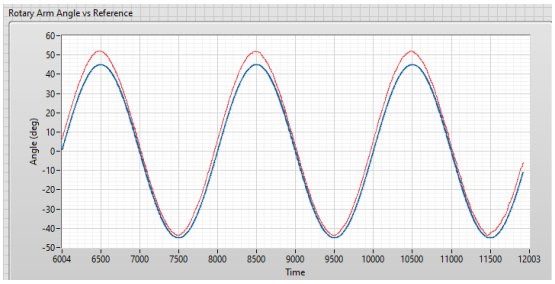


Fig. 11. LQR response of the system. The reference trajectory is blue, and the measured angle θ is red.

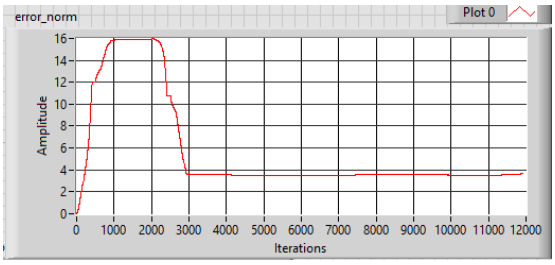


Fig. 12. Norm of tracking error $||\theta_{\text{ref}} - \theta||$ over time of the LQR.

Task B.3 – MPC Control

MPC was implemented as described in III. We implemented the unconstrained, infinite-horizon MPC to minimise the on-line computational cost. For MPC, our state cost matrix was experimentally determined to be:

$$\begin{bmatrix} 80 & 0 & 0 & 0 \\ 0 & 100 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{36}$$

And the control cost was $R = 3$. When the device is turned on, the controller calculates the gain K as the solution to the linear system:

$$FK = G^\top \bar{Q} H \tag{37}$$

Where $F = \bar{R} + G^\top \bar{Q} G$ and $\bar{R}$, G, $\bar{Q}$, and H are as defined in III. Note that it is essential that $\bar{R} + G^\top \bar{Q} G$ is symmetric, so we substitute $(F + F^\top)/2$ for F to ensure it is numerically symmetric. We then apply a control law similar to the one in III and compute the error. The response to the same reference input as LQR (section III) is shown in figure 13.

Task B.4 – ILC Control

First, our plant was stabilised using an LQR controller as described in III. From the original A, B and C matrices, discretised with $F_s = 200$ Hz into A_d, B_d and C_d, then implementing the closed-loop feedback makes our plant model:

$$\begin{aligned} \mathbf{x}(t+1) &= (A_d - B_d K)\mathbf{x} + B_d u \\ y &= \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} \mathbf{x} \end{aligned} \tag{38}$$

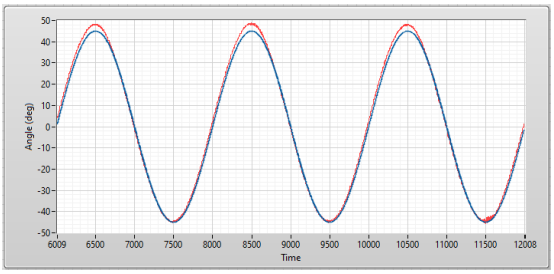


Fig. 13. MPC response of the system. The reference trajectory is blue, and the measured angle θ is red.

Note that C was chosen such that the system output was the arm angle θ. This also converts our plant into an effectively single-input single-output system. We used a PD-type controller as described in III. The error derivative is estimated with the forward-time approximation $\dot{\mathbf{e}}_k \approx \mathbf{e}_k(t+1) - \mathbf{e}_k(t)$. Since the measurements are noisy, Q filtering [1] is used to reject high-frequency noise. This was implemented as a fourth-order Butterworth filter. The actual implementation uses a second-order design with `butter` at a cut-off frequency of $f_c = 2$ Hz, applied with the `filtfilt` function, which applies the filter twice to both the forward and reverse inputs (hence doubling the actual order). It also has zero phase distortion, which improves the feedback performance. Saturation limits are also applied to the optimal input before it is applied to the plant. The parameters chosen were $K_p = 0.05$, $K_d = 0.005$ and $\alpha = 0.95$. The iteration length was $N = 2000$ samples or 10 seconds.

For stability analysis, MATLAB was used for numerical analysis. Q is calculated from the impulse response of the filter, L from rearranging the PD update law into matrix format and P from the impulse response of the closed-loop plant. Note that, since we have a good plant model, we can build P directly from the state-space matrices. From this, $\rho(Q(I - LP)) = 0.9494 < 1$ and hence the system is stable. However, the maximum singular value of the error propagation matrix $\hat{\sigma}(I - PQL) = 1.0913$, so we do not expect this to monotonically converge.

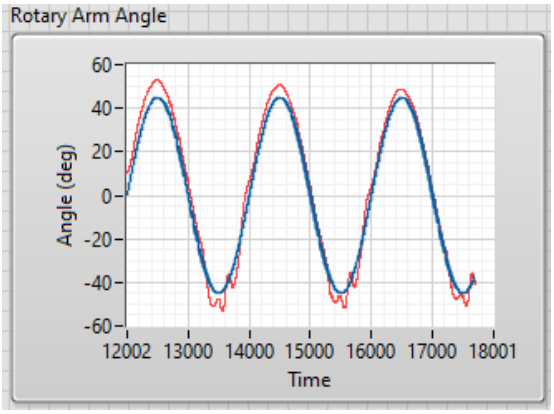


Fig. 14. Pendulum angle (red) vs reference (blue) for ILC tracking on early iterations.

Figure 14 shows the pendulum angle over the reference for early iterations. Clearly, it is not much of an improvement over LQR alone. Figure 15 shows the angle over the reference after a few more iterations, and it has clearly improved. This can be quantified by looking at the tracking error norm over time (see figure 16.

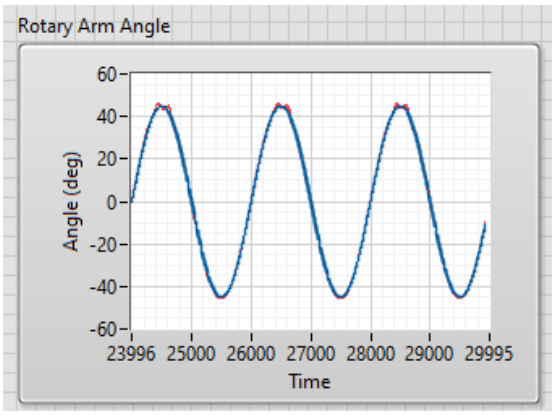


Fig. 15. Pendulum angle (red) vs reference (blue) for ILC tracking on later iterations.

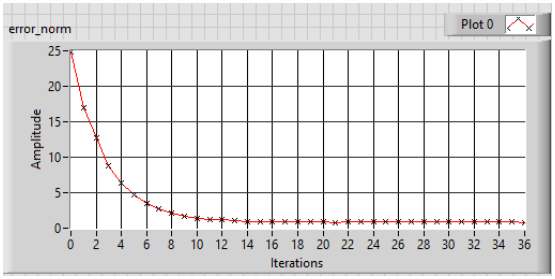


Fig. 16. ILC tracking error over the first 10 iterations, excluding the first.

Figure 17 shows the forward ratio ($|\mathbf{e}_{k+1}|/|\mathbf{e}_k|$) of the error norms. As expected, this does converge towards unity so the scheme does not monotonically converge. However, performance was good, and the ILC controller made significant improvements very quickly.

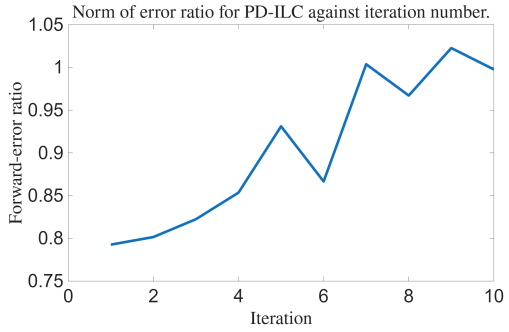


Fig. 17. Ratio of tracking error over iterations.

Then, NOILC was implemented on the pendulum. The same LQR stabilisation scheme used for PD ILC was also used here to allow direct comparisons to be made. q was chosen to be 50. 18 shows the tracking on the first iteration, and also shows the rapid convergence of the input. 19 shows the response on later iterations after the control scheme has had time to come to its steady-state.

Figure 20 shows forward ratio of the error norm, and

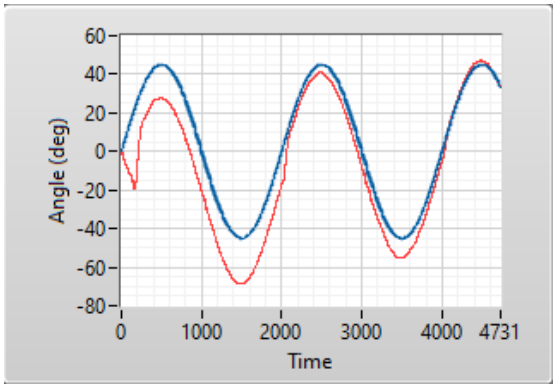


Fig. 18. NOILC θ response vs reference point on the first iteration.

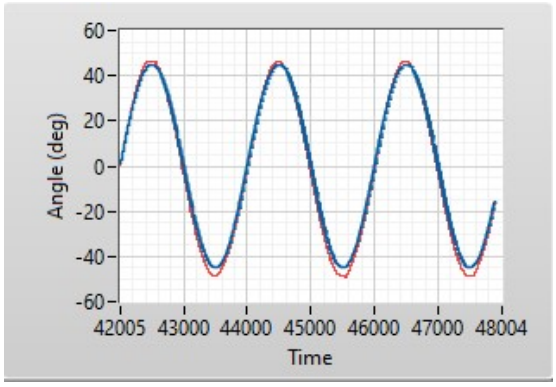


Fig. 19. NOILC θ response vs reference point on later iterations.

illustrates the more rapid convergence compared to PD-ILC.

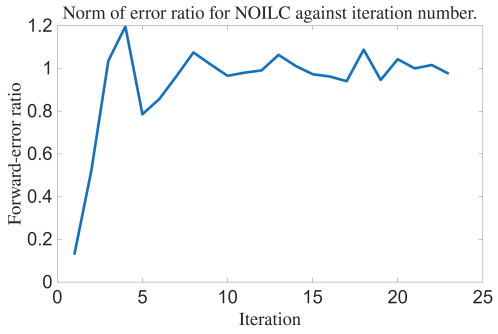


Fig. 20. NOILC error norm ratio over iterations

Task B.5 – Comparison and Discussion

We successfully used all three controller types to stabilise the pendulum in the upright position and track the $45°$, $5s$ sinusoidal reference. Tables II and III summarise the steady-state tracking error norm $\|\theta_{\text{ref}} - \theta\|$ averaged over the final five reference cycles.

In terms of the LQR and MPC implementations, they produced very similar performance. The unconstrained MPC gain in (15) approaches the infinite-horizon LQR gain in (12) as the horizon N tends to infinity; with our horizon and weighting choices the two gains differ only at the third decimal place. Both controllers, therefore, contain similar closed-loop poles and reject disturbances with the same bandwidth. Small residual differences between Fig. 12 and the MPC equivalent come from numerical conditioning of the lifted matrix solve and from slightly heavier velocity penalties used in the MPC weights.

TABLE II
COMPARISON OF BASELINE FEEDBACK CONTROL STRATEGIES.

Feature	**LQR**	**MPC**
Steady-state error norm	∼4	∼2
Computation	gain × state	gain × state
Handles constraints?	no	yes*
Requires repetition?	no	no
Tuning parameters	Q, R	Q, R, N

TABLE III
COMPARISON OF ITERATIVE LEARNING CONTROL STRATEGIES (APPLIED ALONGSIDE LQR).

Feature	**PD ILC**	**NOILC**
Steady-state error norm	∼1	∼2
Computation	filter + update online	precomputed offline
Handles constraints?	via saturation	via saturation
Requires repetition?	yes	yes
Iterations to converge to steady state	~ 14	~ 3
Tuning parameters	α, k_p, k_d, Q-filter	q weight

In terms of model-free (PD) vs model-based (norm-optimal) ILC, the model-based ILC converged much more quickly to its steady-state value (within an iteration or two). This is likely because the ILC controller has a model of the plant and can therefore make better guesses about the control input that would minimise the error more quickly. Interestingly, even though the model-free ILC did not make improvements as quickly, the final steady-state error was much lower ($||\mathbf{e}_\infty|| \approx 0.61$ for model-free, $||\mathbf{e}_\infty|| \approx 2$ for model-based). The main advantage of model-free ILC, of course, is that it does not require a system identification step to be successfully implemented.

IV. CONCLUSION AND FUTURE WORK

We implemented LQR, MPC, and ILC on the Quanser QUBE-Servo to stabilise a pendulum and track the reference arm angle.

Even though we achieved good performance with unconstrained, infinite-horizon MPC, future work could implement a constrained, finite-horizon MPC scheme. It could also be advantageous to verify the model matches the plant itself (e.g. through impulse/frequency response experiments).

We were also unable to demonstrate stability for the model-free ILC scheme, despite good results. This could possibly be achieved by tuning the parameters further and base our tuning on simulation results rather than the physical plant.

REFERENCES

[1] D. Bristow, M. Tharayil, and A. Alleyne, "A survey of iterative learning," *Control Systems, IEEE*, vol. 26, pp. 96 – 114, 07 2006.

[2] M. Norrlöf and S. Gunnarsson, "Time and frequency domain convergence properties in iterative learning control," *International Journal of Control*, vol. 75, no. 14, p. 1114–1126, Jan. 2002. [Online]. Available: http://dx.doi.org/10.1080/00207170210159122

[3] N. Amann, D. Owens, and E. Rogers, "Iterative learning control for discrete time systems with exponential rates of rate convergence," *Proceeding of The Institution of Electrical Engineers on Control Theory and Applications*, vol. 143, no. 2, pp. 217–224, 1996, accepted for publication. [Online]. Available: https://eprints.soton.ac.uk/250133/

[4] Quanser Inc., *QUBE-Servo 2 Workbook – State Space Modeling*, Markham, Ontario, Canada, 2019, student v1.1.

APPENDIX A
ILC ERROR PROPEGATION DERIVATION

The update law for ILC with Q-filtering is:

$$\mathbf{u}_{k+1} = Q(\mathbf{u}_k + L\mathbf{e}_k) \tag{39}$$

And by our lifted P matrix $\mathbf{y}_{k+1} = P\mathbf{u}_{k+1} = PQ(\mathbf{u_k} + \mathbf{Le_k})$. Substituting into the error definition $\mathbf{e}_{k+1} = \mathbf{r} - \mathbf{y}_{k+1}$:

$$\mathbf{e}_{k+1} = \mathbf{r} - PQ(\mathbf{u}_k + L\mathbf{e}_k) \tag{40}$$

And given that $P\mathbf{u}_k = \mathbf{r} - \mathbf{e}_k$:

$$\mathbf{e}_{k+1} = \mathbf{r} - PQ((\mathbf{r} - \mathbf{e}_k) + L\mathbf{e}_k) \tag{41}$$

Expanding and rearranging (41) gives the error propagation as:

$$\mathbf{e}_{k+1} = (I - PQL)\mathbf{e}_k + (I - PQ)\mathbf{r} \tag{42}$$

APPENDIX B
CSV IMPORT TO LABVIEW VI

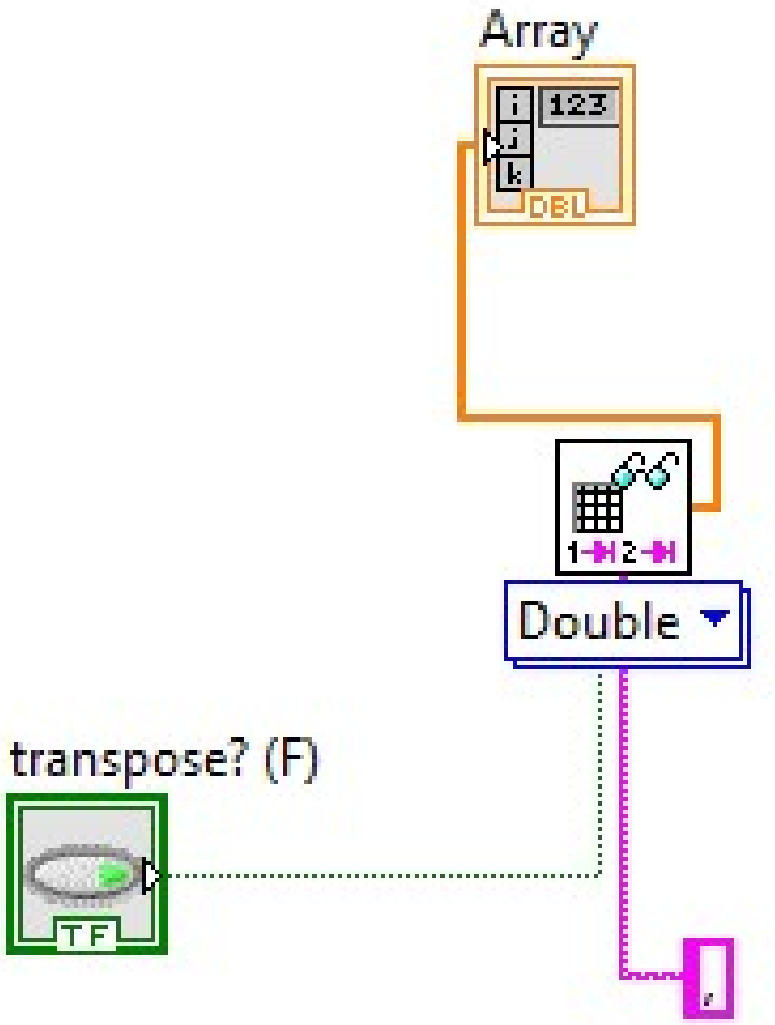


Fig. 21. Figure showing VI used to import CSV file to array in LabVIEW runs on PC, so can select file to convert, array can be converted to constant after run, then copied and pasted into the necessary project

APPENDIX C
ILC CONTROL SYSTEM DESIGN IN LABVIEW

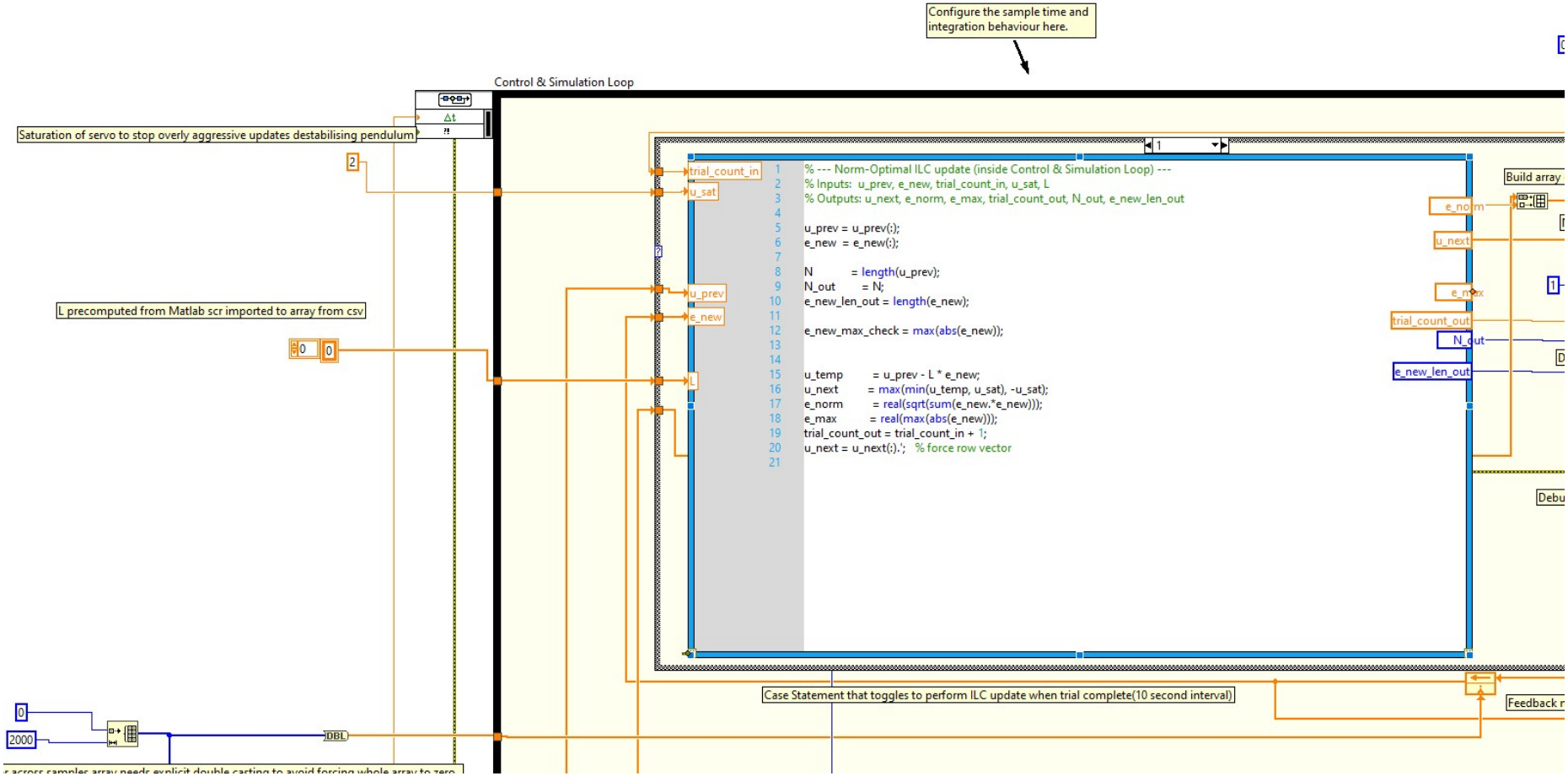


Fig. 22. Structure of Norm Optimal ILC controller design in LabVIEW, the rest of the VI is the same as the PD ILC

APPENDIX D
ILC CONTROL SYSTEM DESIGN IN LABVIEW

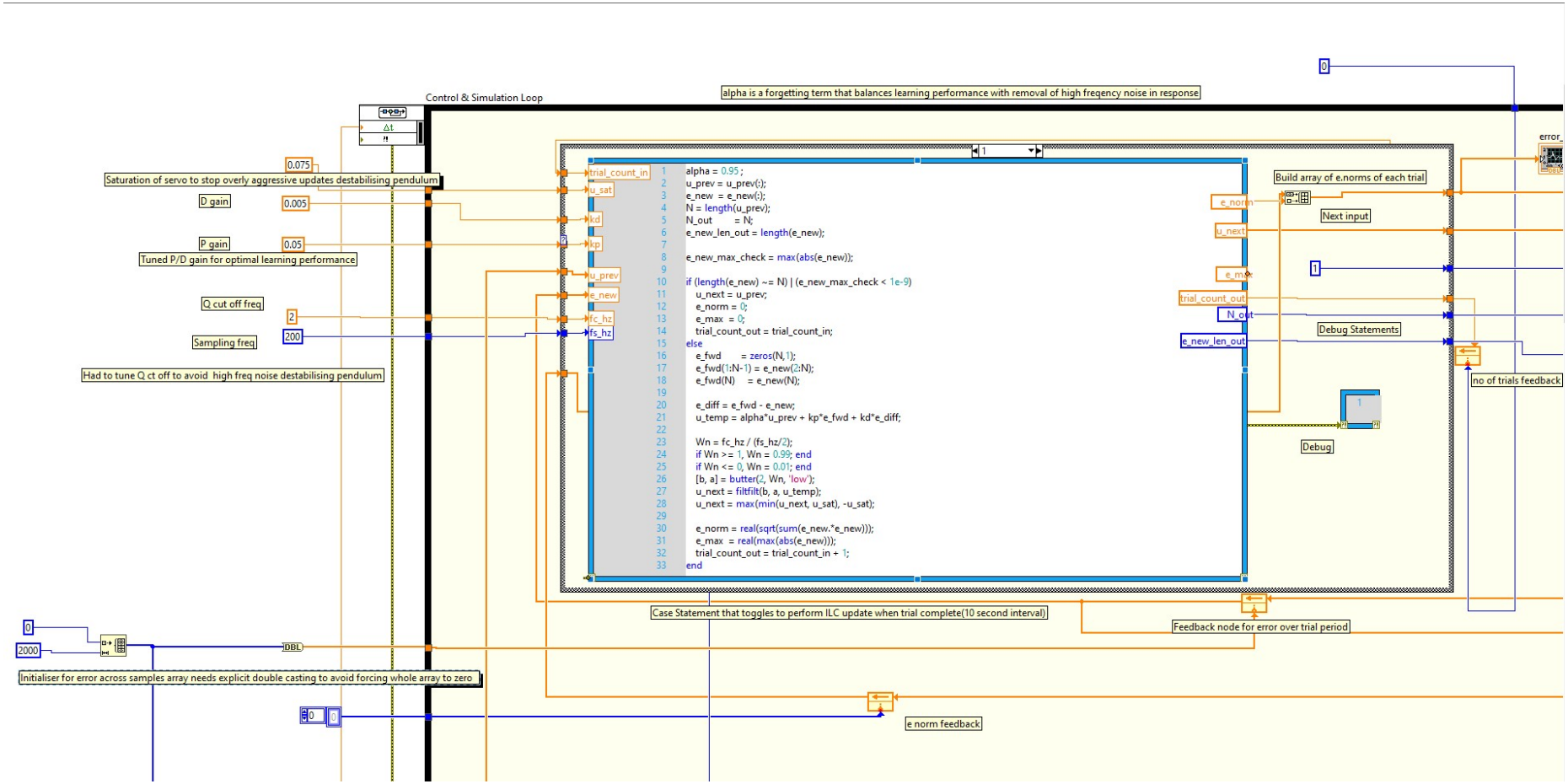


Fig. 23. PD ILC controller design in LabVIEW showing the case statement structure used to trigger the update law at the end of each iteration and the various feedback nodes to pass values across timesteps.

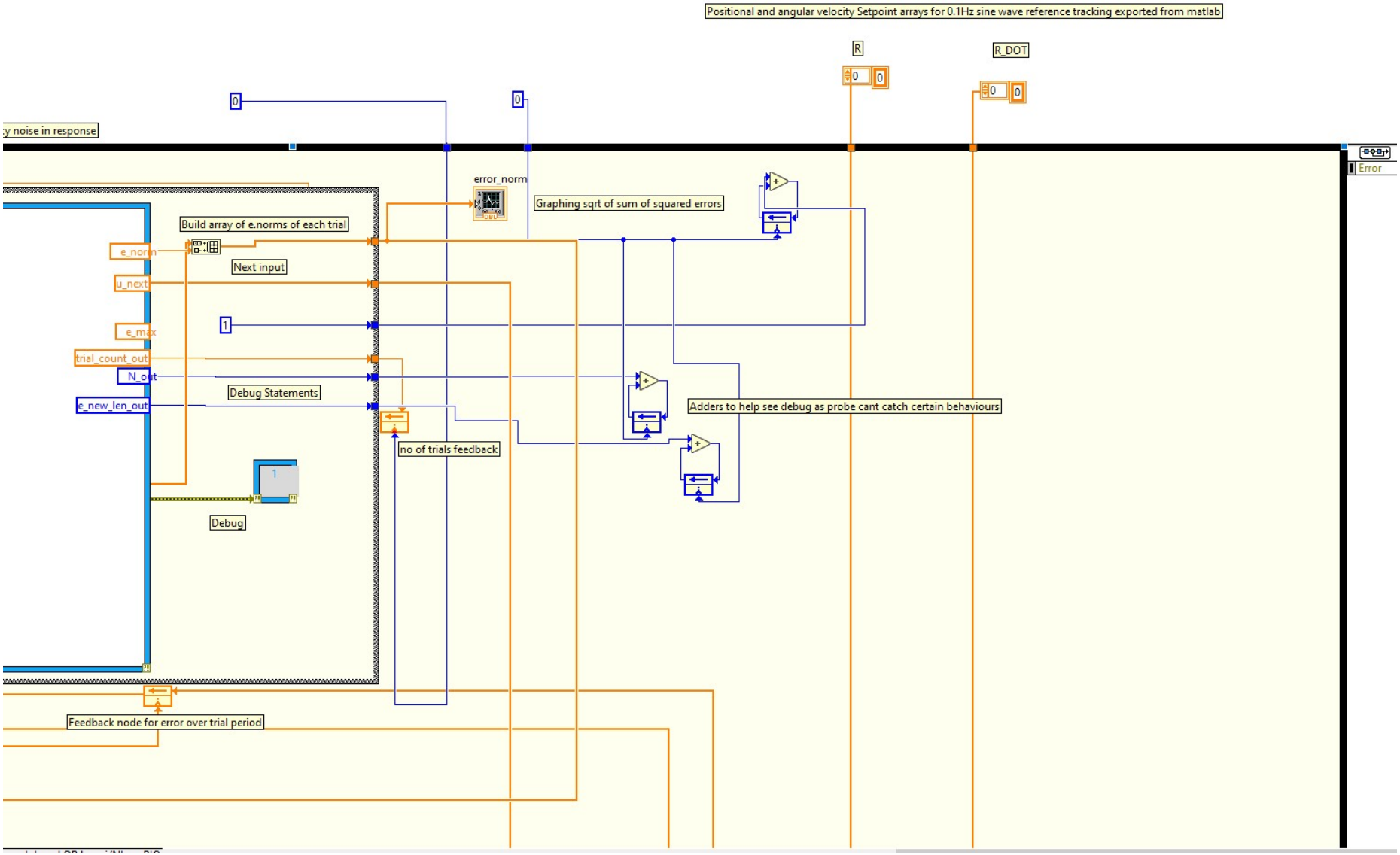

Positional and angular velocity Setpoint arrays for 0.1Hz sine wave reference tracking exported from matlab
R
R_DOT
y noise in response
Error
error_norm
Graphing sqrt of sum of squared errors
Build array of e.norms of each trial
e_norm
Next input
u_next
e_max
trial_count_out
N_out
Debug Statements
e_new_len_out
Adders to help see debug as probe cant catch certain behaviours
no of trials feedback
Debug
Feedback node for error over trial period

Fig. 24.

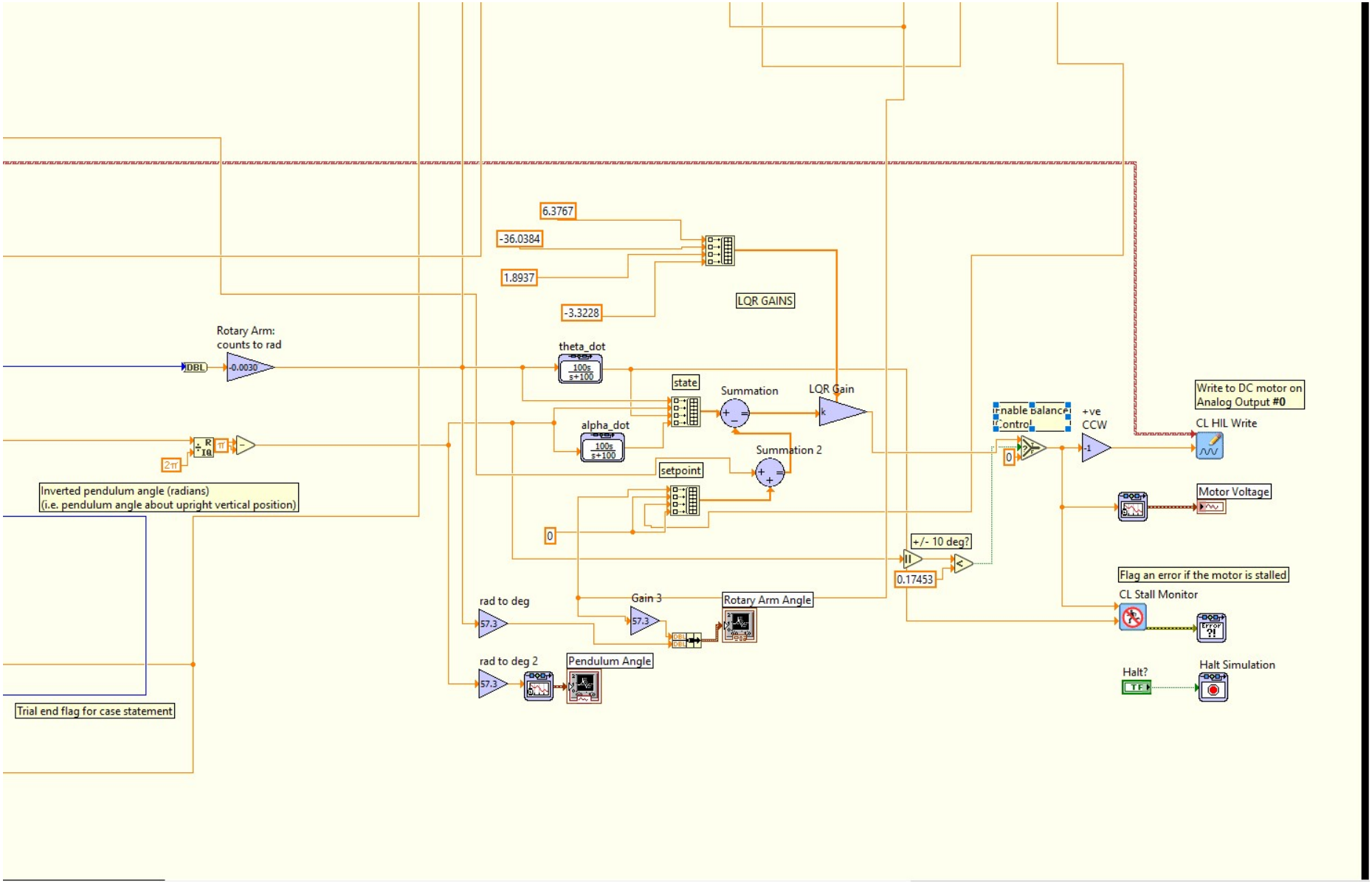

6.3767
-36.0384
1.8937
-3.3228
LQR GAINS
Rotary Arm:
counts to rad
DBL
-0.0030
theta_dot
100s
s+100
state
Summation
LQR Gain
alpha_dot
100s
s+100
Summation 2
setpoint
0
Inverted pendulum angle (radians)
(i.e. pendulum angle about upright vertical position)
Trial end flag for case statement
+/- 10 deg?
0.17453
rad to deg
57.3
Gain 3
57.3
Rotary Arm Angle
rad to deg 2
57.3
Pendulum Angle
Enable Balance
Control
0
+ve
CCW
-1
Write to DC motor on
Analog Output #0
CL HIL Write
Motor Voltage
Flag an error if the motor is stalled
CL Stall Monitor
Halt?
TF
Halt Simulation

Fig. 25.

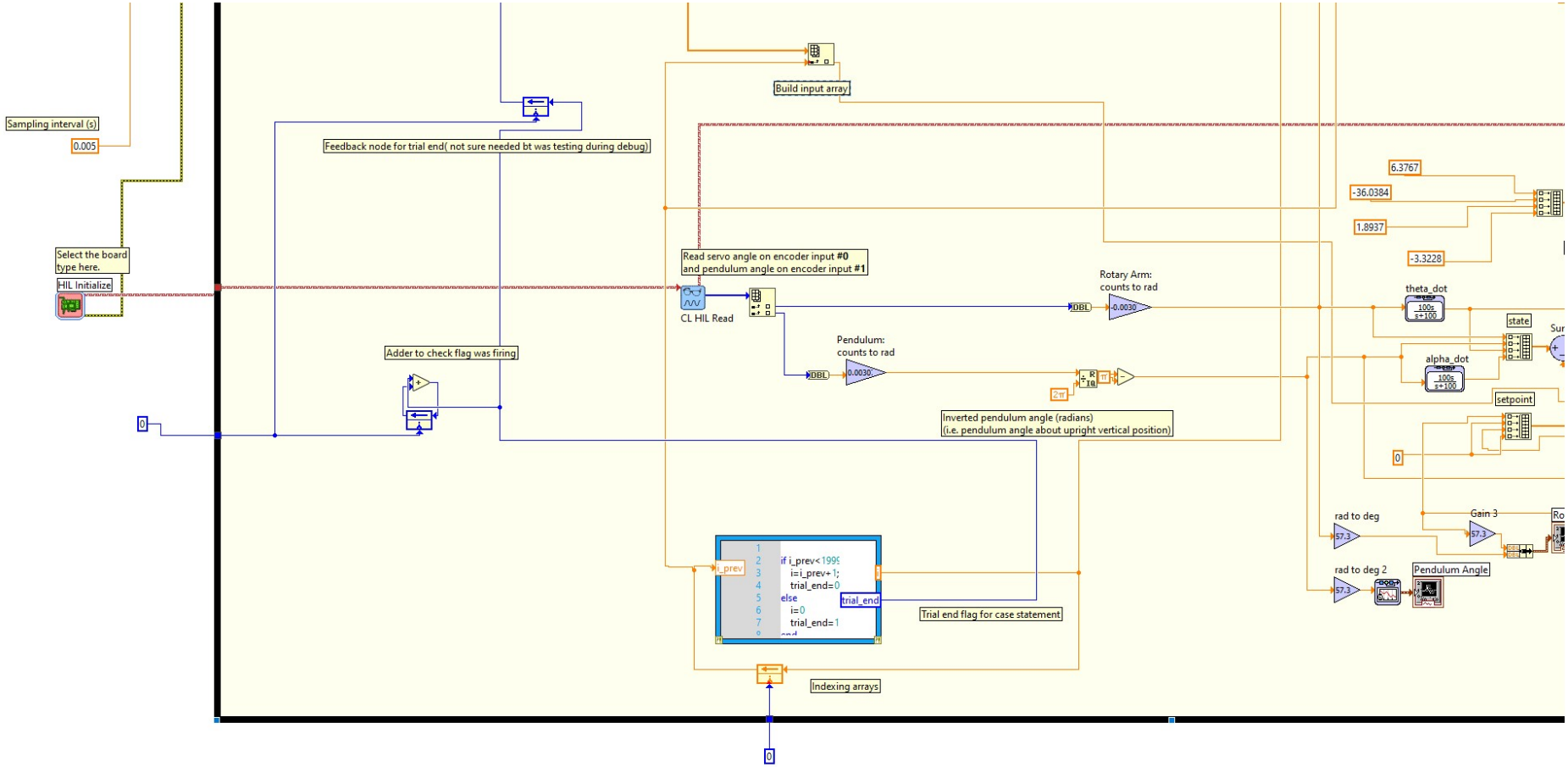

Sampling interval (s)
0.005
Select the board type here.
HIL Initialize
Feedback node for trial end(not sure needed bt was testing during debug)
Build input array
Read servo angle on encoder input #0 and pendulum angle on encoder input #1
CL HIL Read
Rotary Arm: counts to rad
Pendulum: counts to rad
Adder to check flag was firing
Inverted pendulum angle (radians) (i.e. pendulum angle about upright vertical position)
i_prev
trial_end
Trial end flag for case statement
Indexing arrays
6.3767
-36.0384
1.8937
-3.3228
theta_dot
state
alpha_dot
setpoint
rad to deg
Gain 3
rad to deg 2
Pendulum Angle

Fig. 26.

APPENDIX E
MPC CONTROL SYSTEM DESIGN IN LABVIEW

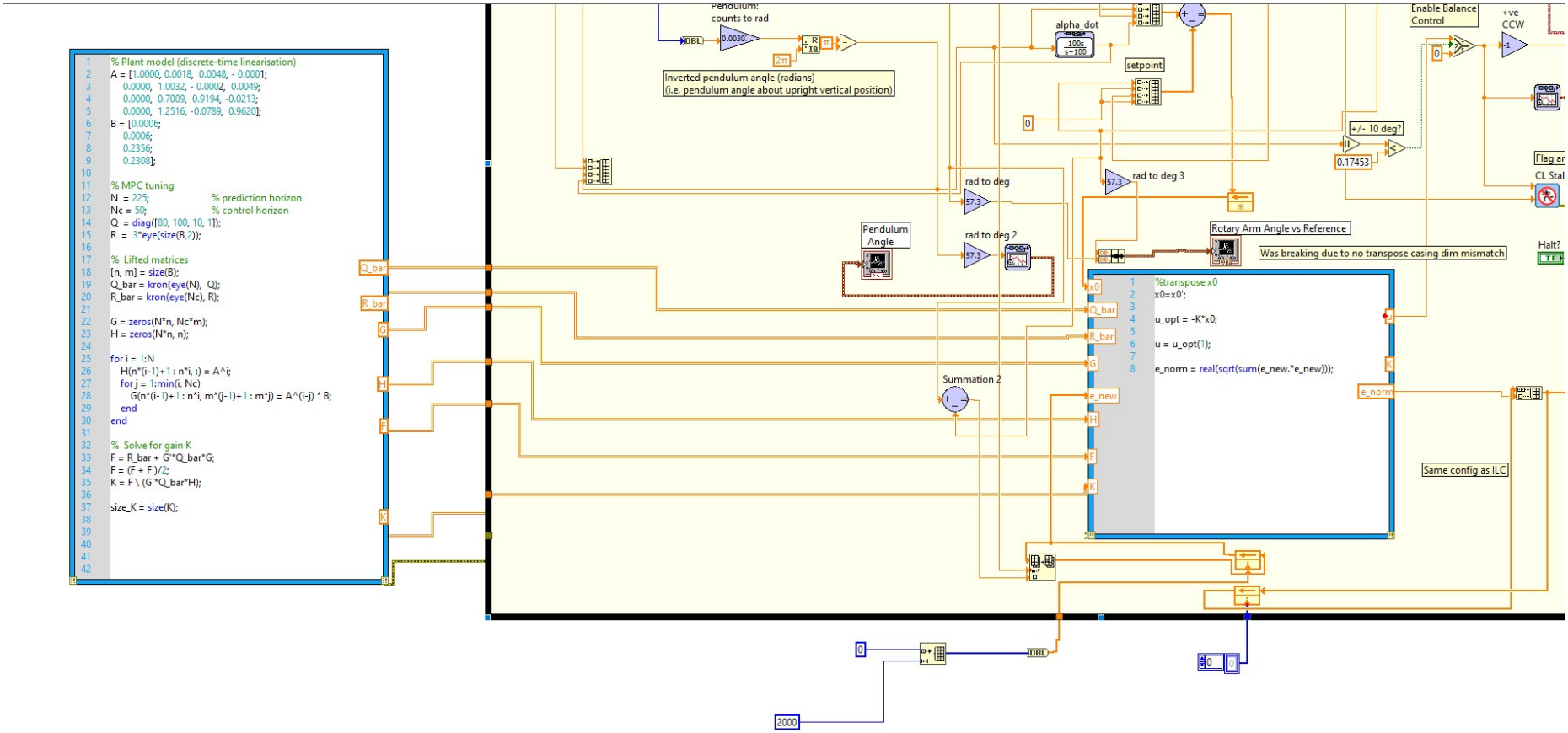


Fig. 27. Structure of MPC controller design in LabVIEW showing how K is precomputed prior to main loop operation and applied at each timestep

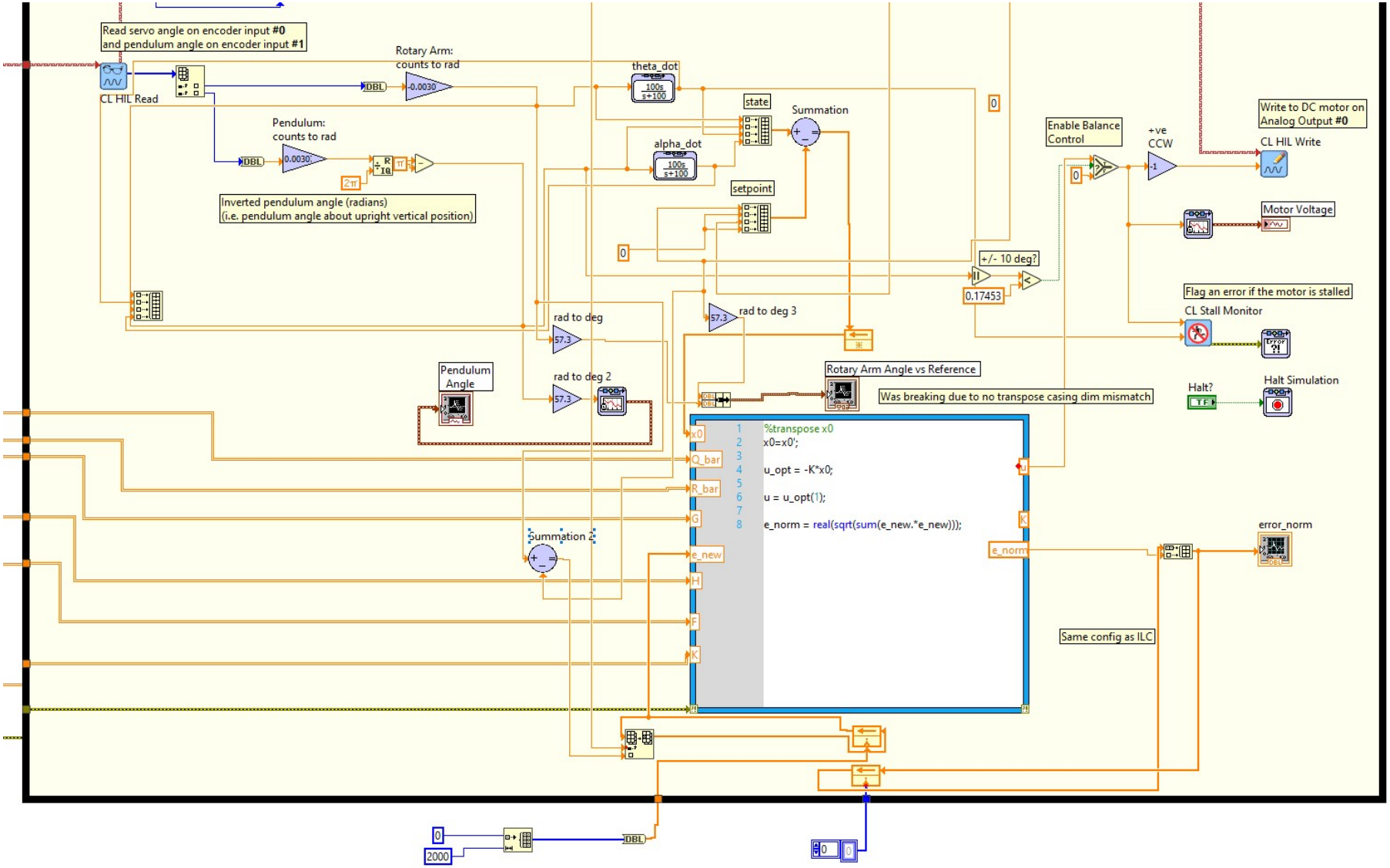

Read servo angle on encoder input #0
and pendulum angle on encoder input #1
CL HIL Read
Rotary Arm:
counts to rad
-0.0030
Pendulum:
counts to rad
0.0030
Inverted pendulum angle (radians)
(i.e. pendulum angle about upright vertical position)
theta_dot
alpha_dot
state
setpoint
Summation
Enable Balance
Control
+ve
CCW
Write to DC motor on
Analog Output #0
CL HIL Write
Motor Voltage
+/- 10 deg?
0.17453
Flag an error if the motor is stalled
CL Stall Monitor
rad to deg
rad to deg 2
rad to deg 3
57.3
Pendulum
Angle
Rotary Arm Angle vs Reference
Was breaking due to no transpose casing dim mismatch
Halt?
Halt Simulation
%transpose x0
x0=x0';
u_opt = -K*x0;
u = u_opt(1);
e_norm = real(sqrt(sum(e_new.*e_new)));
x0
Q_bar
R_bar
G
e_new
H
F
K
u
e_norm
Summation 2
error_norm
Same config as ILC
2000
DBL

Fig. 28.

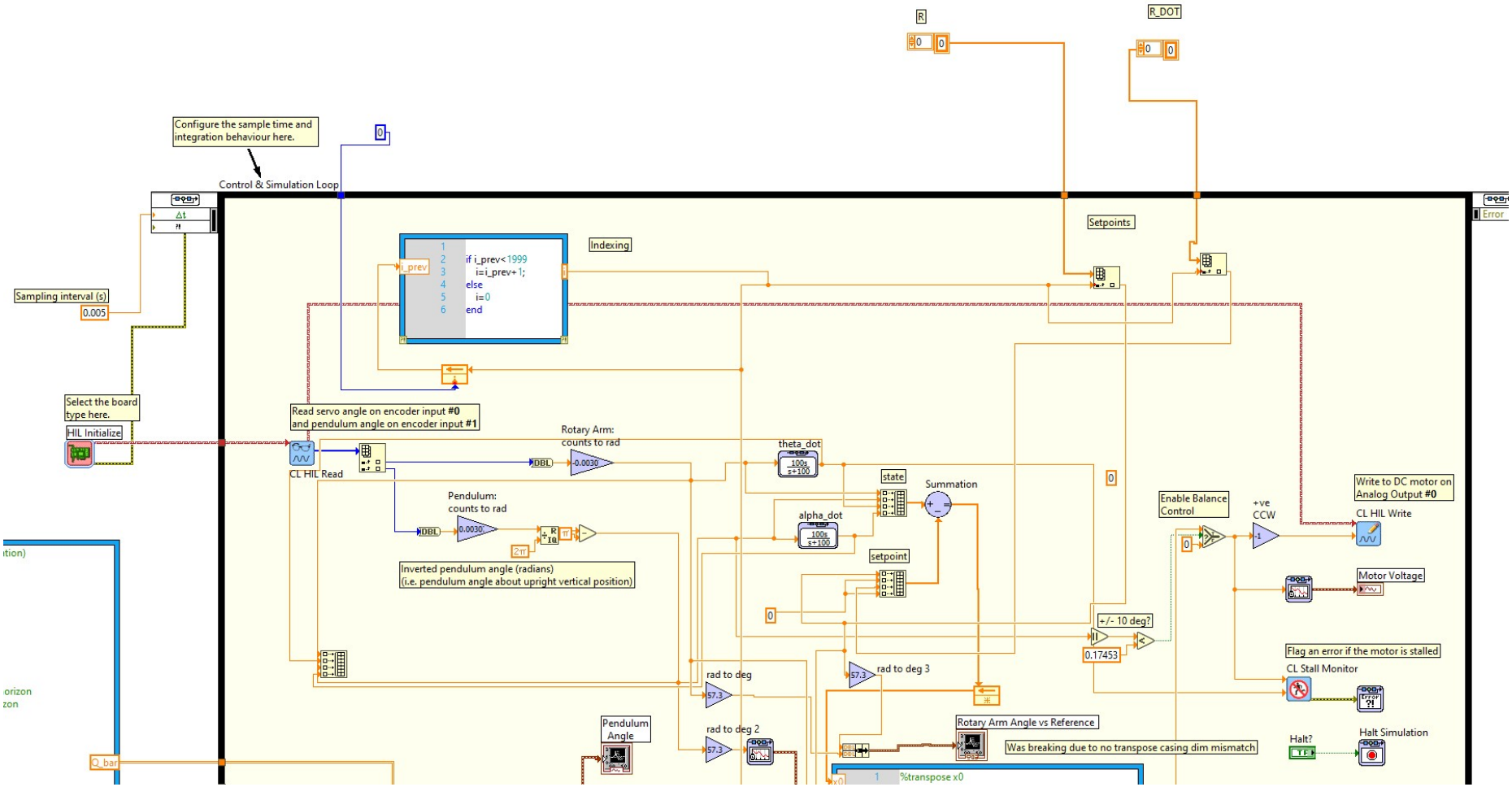

R
R_DOT
Configure the sample time and integration behaviour here.
Control & Simulation Loop
Sampling interval (s)
0.005
Select the board type here.
HIL Initialize
Indexing
if i_prev<1999
i=i_prev+1;
else
i=0
end
Setpoints
Read servo angle on encoder input #0 and pendulum angle on encoder input #1
CL HIL Read
Rotary Arm: counts to rad
-0.0030
Pendulum: counts to rad
0.0030
Inverted pendulum angle (radians) (i.e. pendulum angle about upright vertical position)
theta_dot
alpha_dot
state
setpoint
Summation
Enable Balance Control
+ve CCW
Write to DC motor on Analog Output #0
CL HIL Write
Motor Voltage
+/- 10 deg?
0.17453
Flag an error if the motor is stalled
CL Stall Monitor
rad to deg
rad to deg 2
rad to deg 3
57.3
Pendulum Angle
Rotary Arm Angle vs Reference
Was breaking due to no transpose casing dim mismatch
%transpose x0
Halt?
Halt Simulation
Error

Fig. 29.

APPENDIX F
LQR CONTROL SYSTEM DESIGN IN LABVIEW

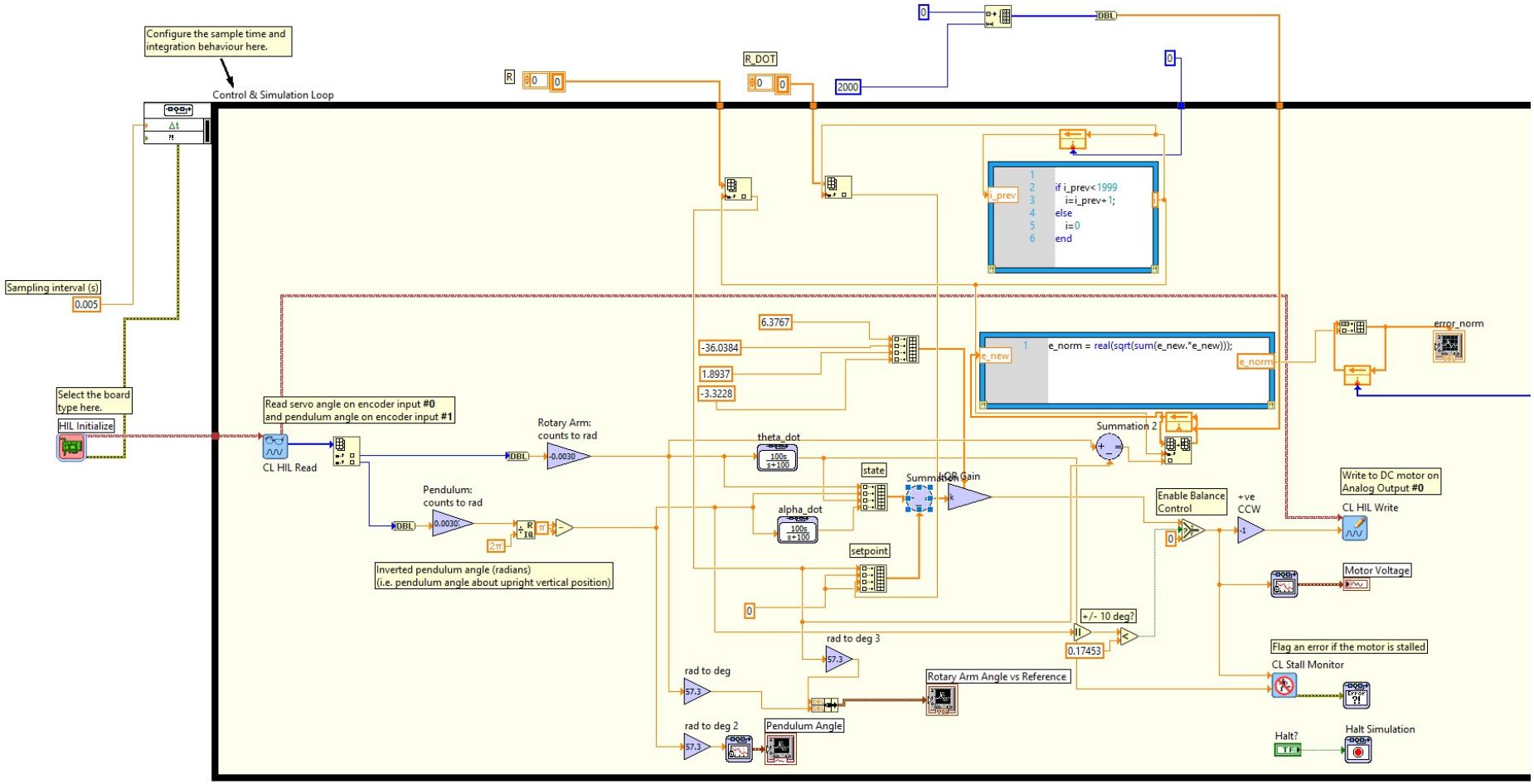


Fig. 30.